

Contents

Implementation Plan: ParaView Output Compaction for Quasi-Dynamic SEAS Runs	2
Overview	2
Constraints	3
Interface constraints	3
Dependency constraints	4
Convention constraints	4
Numerical / performance constraints	5
Project-instructions constraints (from <code>miniapps/seas/CLAUDE.md</code>)	5
Phase 0 (no-code): Audit and quantify	5
Goal	5
Files to Create	5
Files to Modify	5
Detailed Requirements	5
Interfaces	5
Edge Cases	6
Acceptance Criteria	6
Dependencies	6
Phase 1: Gather-to-rank-0 binary appended VTU writer	6
Goal	6
Files to Create	6
Files to Modify	6
Detailed Requirements	6
Interfaces	9
Edge Cases to Handle	9
Acceptance Criteria	10
Dependencies	10
Phase 2: VTKHDF (HDF5) single-file fault output (<i>production target</i>)	10
Goal	10
Sub-phase ordering	10
Phase 2a: Enable HDF5 in the local seas-mfem build (no SEAS code changes)	11
Phase 2b: Fault-submesh + VTKHDF writer integration (<i>local code work</i>)	15
Files to Create	15
Files to Modify	15
Detailed Requirements	16
Interfaces	16
Edge Cases to Handle	17
Acceptance Criteria (Phase 2b — local code work)	17
Dependencies	17
Phase 2c: Replicate the build on Frontera and deploy (<i>after 2a + 2b pass locally</i>)	18
Phase 2d: ZFP lossy compression with separate bulk vs fault tolerances	19
Goal	19
Background — what MFEM exposes today	19
Why per-collection (bulk vs fault) tolerance is natural	20
Sub-phase ordering	20
Phase 2d.1: Install H5Z-ZFP and verify the plugin path	21
Phase 2d.2: Patch MFEM to expose a filter selector + ZFP tolerance	23
Phase 2d.3: Wire the new MFEM API into seas, with separate bulk/fault flags	26
Phase 2d.4: Frontera replication	27
Phase 2d Dependencies	28
Phase 3: Adaptive-schedule snapshot cap and CLI overrides	28
Goal	28
Files to Create	28

Files to Modify	28
Detailed Requirements	28
Interfaces	29
Edge Cases	29
Acceptance Criteria	29
Dependencies	29
Phase 4: Decouple volume PV output from fault PV output	30
Goal	30
Files to Create	30
Files to Modify	30
Detailed Requirements	30
Interfaces	30
Edge Cases	30
Acceptance Criteria	30
Dependencies	31
Phase 5: Post-run packaging and download helpers	31
Goal	31
Files to Create	31
Files to Modify	31
Detailed Requirements	31
Interfaces	31
Edge Cases	31
Acceptance Criteria	32
Dependencies	32
Testing Strategy	32
Per-phase	32
Cross-phase	32
Verification on real benchmarks	32
Risk Assessment	33
What could go wrong	33
Known tricky areas in existing code	34
Open questions to resolve before implementation	34
Baseline measurements (filled in during Phase 0)	34
Acceptance for the plan as a whole	35

Implementation Plan: ParaView Output Compaction for Quasi-Dynamic SEAS Runs

Date: 2026-04-28 (revised 2026-04-29 — added Phase 2d: ZFP lossy compression) **Author:** plan agent (continued from interrupted /code-plan session) **Status:** Draft for review. No code changes proposed in this document — this is the specification a downstream implementer will follow. **Owners:** miniapps/seas/io/paraview_output.hpp (primary), driver call-sites in miniapps/seas/drivers/{tpv102,tpv104,tpv205,seas}__d (secondary). For Phase 2d the ownership extends into MFEM proper: mesh/vtkhdf.{hpp,cpp} and fem/datacollection.{hpp,cpp} (small, local patch). **Production target: VTKHDF single-file output** (Phase 2) with optional **ZFP lossy compression** (Phase 2d) for runs where storage dominates. Phase 1 is the intermediate binary-VTU step; Phase 2 is what production runs use after rollout; Phase 2d is the opt-in storage-saver layered on Phase 2.

Overview

The current ParaView output for quasi-dynamic SEAS simulations (miniapps/seas/io/paraview_output.hpp:WriteFaultS plus the volume pv_.Save() path) emits **one ASCII VTU per MPI rank per write cycle**, plus one

PVTU index per cycle, plus a monotonically growing PVD file. At Frontera-class scale (e.g. 800 ranks $\times$ $\sim$ 5000 writes for a single 250-yr BP5 run) this produces **$\sim$ 4 million tiny files in a single directory**, which:

- breaks Lustre metadata (`du`, `ls`, `tar`, `rsync` all hang);
- cannot be packed on a login node within the 30-min CPU limit;
- compresses poorly because 95% of the data is XML markup;
- is impossible to download to a workstation in any reasonable time.

A live job (job 7672120, FaultSurface dir) just hit **3,894,914 files** and two attempts to `tar` it produced truncated archives (one 312 MB covering only 43,701 files, one 130 MB covering only 4,238 files — both stopped by login-node CPU kills).

This plan rolls out in stages. Phase 1 replaces the per-rank-per-cycle ASCII writer with a gather-to-rank-0 binary appended VTU writer — this works with the current build and gives an immediate $\sim$ 1000 $\times$ file-count reduction. **Phase 2 then enables MFEM’s ParaViewHDFDataCollection (VTKHDF, single-file HDF5)**, which collapses the entire fault time series into one file per run; this is the production target. **Phase 2d (new, 2026-04-29 revision)** layers H5Z-ZFP lossy compression onto Phase 2 with **independent abs-error tolerances for the bulk volume field and the on-fault field**, since their dynamic ranges differ by $\sim$ 6 decades (slip-rate spans 1e-9..1e0 m/s during event nucleation; bulk velocity spans only $\sim$ 3 decades). Phase 3 adds a snapshot-count cap, Phase 4 decouples volume from fault output, and Phase 5 ships a post-run packaging tool for outputs that already exist on disk and predate Phase 1.

Phase 2 requires the seas-mfem build to be reconfigured with `MFEM_USE_HDF5=YES` (and ideally `MFEM_USE_ZLIB=YES`). The plan documents the exact Frontera and local build steps in §Phase 2 so this change can be made by the implementer directly.

Quantitative targets (validated by the unit tests in §Acceptance Criteria):

Metric	Current (job 7672120)	After Phase 1	After Phase 2 (HDF5)	After Phase 2d (HDF5+ZFP)
File count, FaultSurface/	3,894,914	$\sim$ 5,000	1	1
File count per write	n ranks + 1	1	0 (in-place)	0 (in-place)
Bytes / FaultSurface	$\sim$ 250 ASCII	$\sim$ 96 binary	$\sim$ 80	$\sim$ 12–24
VTU per face			HDF5+chunk-zlib	HDF5+ZFP-accuracy
Bulk PV bytes per cycle (TPV102)	$\sim$ few GB ASCII	n/a (Phase 1 fault-only)	$\sim$ 30–50% w/ zlib	$\sim$ 5–10% w/ ZFP @ 1e-3
Time to <code>tar</code> full FaultSurface	unbounded (Lustre-bound)	seconds	N/A — already 1 file	N/A — already 1 file
Time to <code>scp</code> full FaultSurface	unbounded (Lustre-bound)	minutes	seconds	sub-second
Local disk after extract	25 GB+	5 GB	1 GB	0.2–0.4 GB
Min ParaView version	any	5.4	5.11	5.11 (+ H5Z-ZFP plugin)

Constraints

Interface constraints

- The public API of `seas::ParaViewOutput<MeshType>` may extend but **must not break**:
 - `WriteFaultSurfaceVTU(...)` keeps its current 12-positional and 3-optional-positional signature. New behavior is selected via member-state setters or via an alternative entry point

- (e.g. `WriteFaultSurfaceVTUv2`); the existing signature is retained as a thin shim that delegates to the new code path.
- `Save(cycle, time, V_max), ShouldWrite, PeekShouldWrite, CommitSchedule(time), CommitSchedule(time, V_max), ForceSave` — semantics unchanged.
- `SetFaultVTUFields, GetFaultVTUFields` — semantics unchanged (the field-name allow-list keeps applying to the new writer).
- `InitFaultOutputBP5, SetFaultParamsBP5, UpdateFaultFieldsBP5` — semantics unchanged.
- The on-disk **logical layout** that ParaView sees may change (1 VTU/cycle vs N VTUs+1 PVTU), but the time-series PVD must remain readable by ParaView 5.10+. The PVD's `<DataSet timestep="t" file="...">` entries are allowed to point at single VTUs or VTKHDF files — ParaView accepts both.
- The 12 standard fault CellData field names (`slip_dip, slip_strike, slip_rate_dip, slip_rate_strike, traction_dip, traction_strike, state_variable, normal_stress, param_a, param_Dc, fault_x2, fault_x3`) plus the 5 `_k4` diagnostic fields (`slip_rate_dip_k4, slip_rate_strike_k4, traction_dip_k4, traction_strike_k4, normal_stress_k4`) are **byte-identical** to the current writer for any face on any rank, modulo the lossless ASCII↔binary representation change of double-precision values.

Dependency constraints

- **Phase 1 must work with the current build flags** — `MFEM_USE_HDF5 = NO, MFEM_USE_ZLIB = NO` (verified in `config/config.mk:25,43`). No new external libraries for Phase 1. This is the minimum-viable improvement and the bit-exact reference for Phase 2.
- **Phase 2 requires `MFEM_USE_HDF5 = YES` AND** a working parallel HDF5 in the build environment. This is a one-time build change documented in §Phase 2 below. All Phase 2 code is `#ifdef MFEM_USE_HDF5`-guarded so a non-HDF5 build still compiles and runs (the `--paraview-fault-hdf5` flag is rejected at parse time on non-HDF5 builds).
- **`MFEM_USE_ZLIB = YES` is recommended** (not required) so that the Phase 1 binary VTU can use zlib-compressed appended blocks. Without zlib, Phase 1 emits uncompressed binary (still ~5–8× smaller than ASCII). HDF5's internal compression in Phase 2 does NOT depend on this flag.
- **Phase 2d (lossy ZFP) requires the H5Z-ZFP plugin** (LLNL, <https://github.com/LLNL/H5Z-ZFP>, filter ID 32013) plus the underlying `libzfp` (LLNL). The plugin can be loaded dynamically (set `HDF5_PLUGIN_PATH` at runtime) or linked statically into the seas binaries. Phase 2d is **opt-in** per-DataCollection and gated by a runtime CLI flag; the build must additionally define `MFEM_USE_H5Z_ZFP` (new flag introduced by this plan) so non-ZFP builds still compile. When `MFEM_USE_H5Z_ZFP=NO`, Phase 2d code paths are `#ifdef`-guarded out and the corresponding CLI flags are rejected at parse time with a clear error message.
- **MPI** is already a hard dependency of `ParaViewOutput<ParMesh>`. Phase 1 uses `MPI_Gatherv` to rank 0; Phase 2 uses MFEM's collective HDF5 path (all ranks write into the same file). Serial (**Mesh**) builds skip the gather and use a single-rank HDF5 path respectively.

Convention constraints

- Follow the SEAS project conventions per `miniapps/seas/CLAUDE.md`:
 - Do **not** hardcode numerical constants (per `feedback_no_hardcoded_numbers.md`); derive sizes from `n_int, n_shared, nb_f_per_face_`.
 - **Audit ALL occurrences** of a pattern before changing (per `feedback_complete_sign_sites.md`). In particular: every test that opens a written VTU and parses it must be updated alongside the writer change.
 - Drivers must call collectively (all ranks) — the gather is `MPI_Allgatherv` size-collective followed by `MPI_Gatherv` data-collective.
 - File-creation collisions: `mkdir(prefix + "/FaultSurface", 0755)` is rank-0-only with `MPI_Barrier` after, as today (`paraview_output.hpp:740–749`).

Numerical / performance constraints

- Binary appended VTU representation must be **bit-exact** to the current ASCII representation modulo `setprecision(10)` truncation. Switching to binary `double` (8 bytes, IEEE-754) is a **STRICT INCREASE** in precision over `setprecision(10)` (~3 sig figs lost in ASCII). This is desirable, but every test that compares VTU output against a reference must be updated to read binary or to use loose tolerance.
- Gather-to-rank-0 must not block the simulation forward progress. At 800 ranks $\times$ ~5000 fault triangles total, the gather payload per write is $\sim 5e3 \times 17 \text{ fields} \times 8 \text{ B} = \sim 700 \text{ kB/cycle}$ aggregated to rank 0. This is bandwidth-trivial; the cost is in `MPI_Gatherv` collective sync. Observable cost target: **< 5 ms per write at 800 ranks**, measured in Phase 1 acceptance test.
- Write cost: rank 0 doing one `fwrite(...)` of a few MB into an `ofstream` is ~100 μ s on Lustre. Negligible compared to the gather.

Project-instructions constraints (from `miniapps/seas/CLAUDE.md`)

- Files requiring extreme care include `solver/seas_operator.hpp` and `solver/time_stepper.hpp`. This plan does **not** touch them.
 - The schedule cap (Phase 2) interacts with `output_every_n_steps`, `fixed_dt`, and `AdaptiveSchedule` — all three precedence rules must be preserved exactly as documented in `paraview_output.hpp:897-922,928-952,964-`
-

Phase 0 (no-code): Audit and quantify

Goal

Establish the exact baseline file-count and size for one production-class quasidynamic run, so Phase 1's reduction can be measured rather than estimated.

Files to Create

None.

Files to Modify

None.

Detailed Requirements

1. From a representative run (e.g. job 7672120 if recoverable, or whichever subsequent BP5/TPV run is the target), record:
 - `find <results>/FaultSurface -type f | wc -l`
 - `find <results>/FaultSurface -name '*.vtu' -printf '%s\n' | awk 'NR<=1000 {s+=$1} END {print s/NR}'` (mean .vtu size, sample 1000)
 - `wc -l <results>/FaultSurface/fault_surface.pvd`
 - `nrank`s from the sbatch (`SLURM_NTASKS`)
2. Record numbers in a baseline section of this document under “Baseline measurements” before any code change is merged.
3. Identify a *small* representative run (≤ 10 ranks, ≤ 100 cycles) for regression tests in subsequent phases.

Interfaces

None.

Edge Cases

- The baseline FaultSurface dir may already be unreadable due to Lustre metadata exhaustion — in that case, use `lfs find ... | wc -l` instead of GNU `find`, and record the failure mode itself as evidence that the change is needed.

Acceptance Criteria

- Baseline measurements table appears in this document with at least: `n_files`, `n_cycles`, `n_ranks`, `mean_vtu_bytes`, `total_dir_bytes`.

Dependencies

- Depends on: nothing.
 - Required by: Phase 1 acceptance test (uses these numbers as the “before” reference).
-

Phase 1: Gather-to-rank-0 binary appended VTU writer

Goal

After this phase, `WriteFaultSurfaceVTU` produces **one binary VTU per write cycle** instead of `n_ranks` ASCII VTUs + 1 PVTU; the PVD continues to work; all 4 unit tests pass; and a new unit test verifies the gather path on `np=4`.

Files to Create

- `miniapps/seas/io/fault_vtu_binary.hpp` — header-only helpers:
 - VTU XML emit + binary appended block writer.
 - Gather-to-rank-0 of fault geometry and field arrays.
 - Used internally by `paraview_output.hpp`.
- `miniapps/seas/tests/unit/test_fault_surface_vtu_binary.cpp` — verifies bit-exactness vs the ASCII path.
- `miniapps/seas/tests/unit/test_fault_surface_vtu_gather_mpi.cpp` — `mpirun -np 4` test verifying gather+single-file output.

Files to Modify

- `miniapps/seas/io/paraview_output.hpp` — replace the body of `WriteFaultSurfaceVTU` (lines ~571–883) with calls into the new helper. The function’s signature, the field-filter behaviour (`fault_vtu_fields_`), and the `_k4` diagnostic block remain.
- `miniapps/seas/io/Makefile` (if a per-directory Makefile exists; otherwise the top-level `miniapps/seas/Makefile`) — add `test_fault_surface_vtu_binary.o` and `test_fault_surface_vtu_gather_mpi.o` to the test target list.
- `miniapps/seas/tests/unit/test_fault_surface_vtu_continuity.cpp`, `test_fault_surface_vtu_field_filter.cpp`, `test_fault_surface_vtu_k4.cpp`, `test_adjacent_triangle_fault_uniformity.cpp` — switch the VTU parser side from ASCII regex to the new binary-aware reader (helper provided by `fault_vtu_binary.hpp`).

Detailed Requirements

1. Per-rank pack — in `WriteFaultSurfaceVTU`, the existing per-face loop already produces 12 (or 17 with `_k4`) `std::vector<double>` cell-data arrays plus geometry (`vertices`, `triangles`). Pack these into a contiguous byte buffer for gather:

```
// fault_vtu_binary.hpp
namespace mfem::seas::vtu {
```

```

struct LocalFaultPack
{
    // Geometry
    std::vector<std::array<double,3>> vertices;    // 3 doubles per cell
    std::vector<std::array<int,3>> triangles;    // 3 int32 per cell

    // Field arrays, one std::vector<double> per registered field.
    // Names parallel to `field_names`. All arrays have size == triangles.size().
    std::vector<std::vector<double>> field_arrays;
    std::vector<std::string> field_names;
};

} // namespace

```

Building LocalFaultPack is the existing per-face loop; refactor it into a free function BuildLocalFaultPack(...) that takes the same arguments as WriteFaultSurfaceVTU and returns a LocalFaultPack. This isolates the existing logic for per-rank reuse.

2. Gather — variable-length MPI gather to rank 0:

```

// fault_vtu_binary.hpp
namespace mfem::seas::vtu {

struct GatheredFaultPack
{
    std::vector<std::array<double,3>> vertices;    // size = sum_r n_cells_r * 3
    std::vector<std::array<int,3>> triangles;    // size = sum_r n_cells_r,
                                                // indices remapped to global vertex ids
    std::vector<std::vector<double>> field_arrays; // each: size = sum_r n_cells_r
    std::vector<std::string> field_names; // copied from rank 0's local pack
};

/// Collective on `comm`. All ranks must call. Result is populated only
/// on rank 0; on other ranks the returned struct's fields are empty.
GatheredFaultPack GatherFaultPackToRoot(const LocalFaultPack &local,
                                        int rank, int nranks, MPI_Comm comm);

} // namespace

```

Implementation:

- MPI_Allgather of int32_t local_n_cells so every rank knows the per-rank counts (needed for MPI_Gatherv displacement arrays).
- Compute displs[r] = sum_{q<r} counts[q] and total_cells = sum counts.
- For each of the 17 possible field arrays, MPI_Gatherv(local.field_arrays[k].data(), local_n_cells, MPI_DOUBLE, root_buf, counts.data(), displs.data(), MPI_DOUBLE, 0, comm).
- Vertices: every rank emits 3 * n_cells (x,y,z) triples. Gather as MPI_DOUBLE-typed vectors of size 3 * 3 * n_cells (3 verts × 3 coords × n_cells).
- Triangle indices: each rank's local triangle indices (3*i, 3*i+1, 3*i+2) get rebased on rank 0 to (3*(displs_cells[r]+i), 3*(displs_cells[r]+i)+1, ...) so global vertex ids match the gathered vertex layout.
- Field-name list: rank 0 trusts its own list; the data layout is required to match. Add a debug-only assertion MFEM_ASSERT(local.field_names == rank0.field_names) via an MPI_Bcast of a hash of the field-name vector; on mismatch, MFEM_ABORT with the rank id. This guards against driver-side

desync.

3. Binary appended VTU emit — rank 0 writes one VTU using the VTK binary appended encoding (no zlib needed, MFEM_USE_ZLIB=NO). Format spec:

```
<VTKFile type="UnstructuredGrid" version="1.0" byte_order="LittleEndian"
  header_type="UInt64">
  <UnstructuredGrid>
    <Piece NumberOfPoints="..." NumberOfCells="...">
      <Points>
        <DataArray type="Float64" NumberOfComponents="3"
          format="appended" offset="0"/>
      </Points>
      <Cells>
        <DataArray type="Int32" Name="connectivity"
          format="appended" offset="01"/>
        <DataArray type="Int32" Name="offsets"
          format="appended" offset="02"/>
        <DataArray type="UInt8" Name="types"
          format="appended" offset="03"/>
      </Cells>
      <CellData>
        <DataArray type="Float64" Name="slip_dip"
          format="appended" offset="04"/>
        ... one per field ...
      </CellData>
    </Piece>
  </UnstructuredGrid>
  <AppendedData encoding="raw">
    _<8-byte little-endian uint64 length><N bytes><...next array...>
  </AppendedData>
</VTKFile>
```

The leading `_` marker after `<AppendedData encoding="raw">` is required by the VTU spec. Each array's “block” is: `[uint64 byte_length][raw_bytes]`. `header_type="UInt64"` (not the default `UInt32`) future-proofs against the >4 GB threshold.

The offsets 01..017 are computed by the writer as the running byte position **including** the 8-byte length headers for each preceding block.

4. PVD time-series — semantics unchanged. The PVD now points at the single per-cycle VTU instead of the per-cycle PVTU. The PVTU writer (rank 0, lines 836–882) is removed entirely. PVD entry format:

```
<DataSet timestep="0.0000000000000000e+00"
  file="fault_surface_c0.vtu"/>
```

(rank-suffixed `_r0` from the old per-rank scheme is dropped — there are no per-rank files anymore.)

5. Backward-compat ASCII fallback — add a build-time-or-runtime toggle `kEnableLegacyAsciiVTU` (default `false`). Drivers that need the legacy ASCII format for debugging can call `pv_out->SetLegacyAsciiVTU(true)` to revert to the old per-rank ASCII writer. Keep the legacy path as a `static` private member function so it can be deleted after Phase 4 stabilises.

6. Field filter — `fault_vtu_fields_` (the `SetFaultVTUFields` allow-list) applies `UNCHANGED` in the new path: the `BuildLocalFaultPack` function reads `fault_vtu_fields_` and skips arrays not in the list. Because gathering empty arrays is wasteful, the set of gathered arrays is pruned at the **LocalFaultPack** level, not just at write time.

Interfaces

```
// New, in fault_vtu_binary.hpp
namespace mfem::seas::vtu {

LocalFaultPack BuildLocalFaultPack(
    const Mesh &mesh, // or ParMesh
    const Array<int> &fault_interior_faces,
    const Array<int> &fault_shared_faces,
    int nbf_per_face,
    const Vector &local_slip, const Vector &local_slip_rate,
    const Vector &local_traction, const Vector &local_state,
    const Vector &local_normal_stress,
    const Vector &local_a, const Vector &local_Dc,
    const Vector &local_x2, const Vector &local_x3,
    const Vector &local_slip_rate_k4, const Vector &local_traction_k4,
    const Vector &local_normal_stress_k4,
    const std::set<std::string> &field_filter);

GatheredFaultPack GatherFaultPackToRoot(
    const LocalFaultPack &local,
    int rank, int nranks, MPI_Comm comm);

void WriteFaultPackVTU(const std::string &vtu_path,
    const GatheredFaultPack &g);

} // namespace
```

Inside paraview_output.hpp, the body of WriteFaultSurfaceVTU is replaced by:

```
auto local = vtu::BuildLocalFaultPack(...);
MPI_Comm comm = MPI_COMM_NULL;
if constexpr (std::is_same_v<MeshType, ParMesh>) { comm = mesh_.GetComm(); }
auto gathered = vtu::GatherFaultPackToRoot(local, rank, nranks, comm);
if (rank == 0)
{
    ::mkdir((prefix + "/FaultSurface").c_str(), 0755);
    const std::string vtu_path = prefix + "/FaultSurface/fault_surface_c"
        + std::to_string(cycle) + ".vtu";
    vtu::WriteFaultPackVTU(vtu_path, gathered);
    fault_pvd_entries_.push_back({time, "fault_surface_c"
        + std::to_string(cycle) + ".vtu"});
    WriteFaultPVD(prefix + "/FaultSurface");
}
}
```

Edge Cases to Handle

- **Single-rank serial build** (MeshType = Mesh): GatherFaultPackToRoot is a no-op that copies local straight into a GatheredFaultPack and returns. Verified by test_fault_surface_vtu_binary.cpp.
- **Empty fault on a rank**: Some ranks have zero local fault faces. MPI_Gatherv with a per-rank count of zero is well-defined as long as the buffer pointer is non-NULL on root and may be NULL on the empty rank. Pass nullptr from empty ranks; verify via valgrind in the unit test on np=4.
- **nbfs_per_face == 0**: existing MFEM_VERIFY check at line 599 preserved at the top of BuildLocalFaultPack.
- **All field filter rejects all fields**: the writer must still emit <Points> and <Cells> so ParaView can render geometry. <CellData> is emitted but empty (no <DataArray> children). Test in test_fault_surface_vtu_field_filter.cpp (extend the existing test).

- **_k4 diagnostic mode:** when the three `_k4` vectors are non-empty, the 5 extra arrays are added to `LocalFaultPack.field_arrays` exactly as today. No per-rank desync risk because all ranks see the same `local_*_k4.Size() > 0` condition (driver-side invariant).
- **Endianness:** `byte_order="LittleEndian"` is hardcoded in the VTU. All TACC clusters are x86-64 little-endian. If the build ever moves to a big-endian host, the writer must byte-swap before writing AND emit `byte_order="BigEndian"`. Add a `static_assert(__BYTE_ORDER__ == __ORDER_LITTLE_ENDIAN__, ...)` in `WriteFaultPackVTU` for now.

Acceptance Criteria

- ☐ `make test-fault-surface-vtu` passes (existing 4 unit tests + 2 new ones).
- ☐ On a representative serial run, the new VTU is byte-readable by ParaView 5.10 and renders identical geometry/fields to the old ASCII output (visual diff, plus per-field max-abs-diff < 1e-9 verified by `test_fault_surface_vtu_binary.cpp`).
- ☐ On `np=4`, the new VTU written by all-ranks-collective call is byte-identical to the result of running serially with the same fault data (see `test_fault_surface_vtu_gather_mpi.cpp`).
- ☐ File count in `FaultSurface/` after a 100-cycle run on `np=4` is exactly **101** (100 VTUs + 1 PVD), not the old $100 * 4 + 100 + 1 = 501$.
- ☐ Per-cycle `FaultSurface` bytes (mean over 100 cycles) is < 50% of the old ASCII baseline measured in Phase 0.
- ☐ Total `WriteFaultSurfaceVTU` wall-time per call on `np=4` is < $2\times$ the old per-rank ASCII writer's average wall-time. (Gather adds collective cost; we accept up to $2\times$ per-call wall-time in exchange for the file-count reduction.)
- ☐ `make test-bp5-smoke` (`mpirun -np 4 seas_test_bp5_parallel_smoke`) passes.
- ☐ `make test-tpv102` and `make test-tpv104` pass.

Dependencies

- Depends on: Phase 0 baseline measurements.
- Required by: Phase 2 (the snapshot cap is observed by the new writer); Phase 4 (HDF5 mode reuses `LocalFaultPack/GatheredFaultPack`).

Phase 2: VTKHDF (HDF5) single-file fault output (*production target*)

Goal

After this phase, a single-file `output_dir/fault_surface.vtkhdf` contains the entire fault time-series for a run. This is the recommended production mode at TACC scale: 1 file, 1 download, 1 extraction. Drivers default to this mode when the build supports HDF5 and the user has not explicitly requested binary VTU.

Sub-phase ordering

To keep risk low, Phase 2 is split into three sub-phases that **must be executed in order**. Each sub-phase is independently acceptance-tested and a gate for the next.

Sub-phase	What	Where	Acceptance gate
2a	Enable HDF5 + zlib in the local seas-mfem build (mfem-dev conda env on macOS arm64); verify ParaViewHDFDataCollection round-trips a tiny dataset.	Local Mac	<code>seas_test_paraview_hdf_smoke</code> passes serial and <code>mpirun -np 4</code> .
2b	Implement the fault-submesh + HDF5 writer integration in <code>paraview_output.hpp</code> ; bit-exactness vs Phase 1 binary VTU.	Local Mac	<code>seas_test_fault_surface_vtkhdf</code> and <code>_vtkhdf_mpi</code> pass; <code>make test-fault-surface-vtu-binary</code> still passes (regression gate).
2c	Replicate the same recipe on Frontera; submit a small BP5 microrun (≤ 1 yr, ≤ 100 ranks) producing a single <code>fault_surface.vtkhdf</code> .	Frontera	One file in <code>FaultSurface/</code> ; ParaView 5.11+ on workstation opens and scrubs through all timesteps.

The local-first ordering catches build / linkage / API problems on a machine the implementer can iterate on in seconds, before paying the ~10 minute cycle time of a Frontera rebuild.

Phase 2a: Enable HDF5 in the local seas-mfem build (no SEAS code changes)

2a.1 Local environment baseline (verified 2026-04-28)

Item	Value
Conda env	<code>/Users/chunhuizhao/miniforge/envs/mfem-dev</code>
Compiler	<code>clang 19.1.7</code> from miniforge (target <code>arm64-apple-darwin20.0.0</code>)
MPI	OpenMPI (<code>libmpi.40.dylib</code> $\Rightarrow$ <code>mpi_openmpi_*</code> HDF5 build is the right flavor)
mpicxx	<code>/Users/chunhuizhao/miniforge/envs/mfem-dev/bin/mpicxx</code>
zlib	already installed
HDF5	(<code>libz.{a,1.dylib,1.3.1.dylib}</code>) not installed (no <code>h5cc</code> , no <code>libhdf5*</code> , no <code>hdf5*.h</code>)
ParaView	not installed (no <code>paraview</code> , no <code>pvpython</code>)
<code>config/config.mk</code>	<code>MFEM_USE_HDF5 = NO</code> , <code>MFEM_USE_ZLIB = NO</code>

The implementer verifies these with the commands listed under “Acceptance” below before proceeding.

2a.2 Install parallel HDF5 (MPI flavor must match)

```
conda activate mfem-dev
```

```
# CRITICAL: pick the OpenMPI flavor of HDF5 to match this env's MPI.
```

Installing the MPICH flavor links but fails at MPI runtime.

```
conda install -y -c conda-forge "hdf5==*openmpi*"
```

Verify install

```
which h5pcc h5dump
```

```
ls /Users/chunhuizhao/miniforge/envs/mfem-dev/lib/libhdf5*
```

Confirm the same MPI library is being linked

```
otool -L $(which h5pcc) 2>&1 | grep mpi
```

```
otool -L /Users/chunhuizhao/miniforge/envs/mfem-dev/lib/libhdf5.dylib | grep mpi
```

*# Both must show /Users/chunhuizhao/miniforge/envs/mfem-dev/lib/libmpi.**

If `conda install` resolves to a non-OpenMPI build (e.g. it pulls MPICH and replaces the env's MPI), abort and use the explicit version spec:

```
conda install -y -c conda-forge "hdf5=1.14.*=mpi_openmpi_*"
```

2a.3 (Optional but recommended) Install ParaView for end-of-pipe verification

Option A - Mac app bundle (preferred, full GUI)

```
brew install --cask paraview # or download the .dmg from paraview.org
```

Option B - pyvista for programmatic VTKHDF inspection

```
conda install -y -c conda-forge pyvista 'h5py>=3.8'
```

ParaView 5.11+ is required to open VTKHDF files. brew's current cask is 5.13 at the time of writing. If neither install is feasible, the test suite (Phase 2b) still proves correctness — but visual inspection is the only end-to-end check that ParaView's HDF reader and the emitted file agree.

2a.4 Edit config/config.mk Replace lines 25 (zlib) and 43 (hdf5) with their YES counterparts and add the HDF5_DIR / library-path block. Concrete diff:

```
# Use the corresponding settings to enable/disable each library.
```

```
-MFEM_USE_ZLIB          = NO
```

```
+MFEM_USE_ZLIB          = YES
```

```
# Use HDF5 library to enable reading/writing %VTKHDF files.  Requires HDF5
# development headers and library.
```

```
-MFEM_USE_HDF5          = NO
```

```
+MFEM_USE_HDF5          = YES
```

```
# Configuration for HDF5 - point at the conda env (mfem-dev) prefix.  The
# parallel HDF5 build installed by conda-forge with mpi_openmpi_* selector
# places libhdf5.dylib, libhdf5_hl.dylib, and the headers under
# $(CONDA_PREFIX)/lib and $(CONDA_PREFIX)/include respectively.
```

```
+HDF5_DIR               = $(CONDA_PREFIX)
```

```
+HDF5_OPT               = -I$(HDF5_DIR)/include
```

```
+HDF5_LIB               = -L$(HDF5_DIR)/lib -lhdf5_hl -lhdf5
```

Note: the existing `defaults.mk:HDF5_DIR = $(HOME)/local` is a placeholder and is overridden by the explicit assignment above.

For zlib the conda env's `-lz` is already on the system link path because `mpicxx` is from the same conda env; no `ZLIB_DIR` override is required. If a future linker error references `libz`, add:

```
ZLIB_DIR = $(CONDA_PREFIX)
```

2a.5 Rebuild MFEM and the seas miniapp

```
cd /Users/chunhuizhao/projects/seas-mfem
```

```
# 1. Re-derive config headers (config/_config.hpp, config/config.mk)
make config

# 2. Sanity-check that config.hpp now defines the macros
grep -E 'define MFEM_USE_HDF5|define MFEM_USE_ZLIB' config/_config.hpp
# both expected to print

# 3. Rebuild MFEM static lib (this takes ~5-10 minutes on M-series Mac).
#   A clean rebuild of just libmfem.a (not the entire test/example tree).
make clean && make -j 8 lib

# 4. Rebuild only the SEAS targets we care about for the smoke test.
cd miniapps/seas
make seas_bp5_full -j 8
```

Common build issues and resolution: - 'hdf5.h' file not found → HDF5_DIR not pointing at conda env; verify \$(CONDA_PREFIX) expands to the mfem-dev env at make time. - Undefined symbols for architecture arm64: H5Fcreate → linker not picking up -lhdf5; check HDF5_LIB was appended to MFEM_EXT_LIBS (defaults.mk:391). - ParaViewHDFDataCollection symbols missing in nm seas_bp5_full | grep -i HDF → MFEM_USE_HDF5=YES did not propagate; double-check make config was rerun and _config.hpp was regenerated.

2a.6 Local smoke test — ParaViewHDFDataCollection round-trip Add a single-file test under miniapps/seas/tests/unit/ that exercises MFEM's HDF5 writer **without** any seas-mfem code path. This isolates the build change from the Phase 2b code change.

File to create: miniapps/seas/tests/unit/test_paraview_hdf_smoke.cpp

```
// 2a.6 smoke test - verify MFEM's ParaViewHDFDataCollection writes and
// reads a single .vtkhdf for a tiny 1-element mesh. No seas-mfem
// dependencies; tests the build-config change only.
#include "mfem.hpp"
#include <iostream>
#ifdef MFEM_USE_HDF5
#include "fem/datacollection.hpp" // ParaViewHDFDataCollection
#include <hdf5.h> // H5_VERS_*
#endif

int main(int argc, char *argv[])
{
    mfem::Mpi::Init(argc, argv);
    const int rank = mfem::Mpi::WorldRank();
#ifdef MFEM_USE_HDF5
    if (rank == 0)
    { std::cerr << "FAIL: MFEM_USE_HDF5 is OFF in this build\n"; }
    return 1;
#else
    if (rank == 0)
    {
        std::cout << "HDF5 version compiled against: "
                    << H5_VERS_MAJOR << "." << H5_VERS_MINOR << "."
                    << H5_VERS_RELEASE << "\n";
    }
}
```

```

}
// Minimal mesh: one tetrahedron, one L2-p0 grid function.
mfem::Mesh smesh = mfem::Mesh::MakeCartesian3D(1, 1, 1,
                                                mfem::Element::TETRAHEDRON);

mfem::ParMesh mesh(MPI_COMM_WORLD, smesh);
mfem::L2_FECollection fec(0, mesh.Dimension());
mfem::ParFiniteElementSpace fes(&mesh, &fec);
mfem::ParGridFunction gf(&fes);
gf = 3.14;

mfem::ParaViewHDFDataCollection dc("smoke", &mesh);
dc.SetPrefixPath("/tmp/test_paraview_hdf_smoke_out");
dc.SetDataFormat(mfem::VTKFormat::BINARY);
dc.SetCompression(true);
dc.SetCompressionLevel(3);
dc.RegisterField("scalar", &gf);
dc.SetCycle(0);
dc.SetTime(0.0);
dc.Save();

if (rank == 0)
{
    std::ifstream f("/tmp/test_paraview_hdf_smoke_out/smoke.vtkhdf",
                    std::ios::binary);
    if (!f.is_open())
    {
        std::cerr << "FAIL: smoke.vtkhdf not created\n"; return 1; }
    char sig[8]; f.read(sig, 8);
    // VTKHDF files start with HDF5 superblock signature 0x89 H D F \r \n 0x1a \n
    bool ok = (sig[0] == '\x89' && sig[1] == 'H' && sig[2] == 'D'
               && sig[3] == 'F' && sig[4] == '\r' && sig[5] == '\n'
               && sig[6] == '\x1a' && sig[7] == '\n');
    if (!ok)
    {
        std::cerr << "FAIL: HDF5 signature missing\n"; return 1; }
    std::cout << "PASS\n";
}
return 0;
#endif
}

```

Makefile target (in miniapps/seas/Makefile):

```

TEST_PARAVIEW_HDF_SMOKE_SRC = tests/unit/test_paraview_hdf_smoke.cpp
TEST_PARAVIEW_HDF_SMOKE_OBJ = $(TEST_PARAVIEW_HDF_SMOKE_SRC:.cpp=.o)

seas_test_paraview_hdf_smoke: $(TEST_PARAVIEW_HDF_SMOKE_OBJ)
    $(MFEM_CXX) $(MFEM_LINK_FLAGS) -o $@ $(TEST_PARAVIEW_HDF_SMOKE_OBJ) $(MFEM_LIBS)

$(TEST_PARAVIEW_HDF_SMOKE_OBJ): %.o: $(SRC)%.cpp $(SEAS_HEADERS) $(MFEM_LIB_FILE) $(CONFIG_MK)
    @mkdir -p $(@D)
    $(MFEM_CXX) $(MFEM_FLAGS) $(SEAS_INCLUDES) -c $< -o $@

test-paraview-hdf-smoke: seas_test_paraview_hdf_smoke
    ./seas_test_paraview_hdf_smoke
    mpirun -np 4 ./seas_test_paraview_hdf_smoke

```

2a Acceptance gate (must pass before Phase 2b)

- `which h5pcc` resolves and `otool -L $(which h5pcc) | grep mpi` shows the `mfem-dev` env's `libmpi`.
 - `grep MFEM_USE_HDF5 config/_config.hpp` confirms the macro is defined (not `// #define`).
 - `make lib -j 8` from the `seas-mfem` root succeeds; `nm lib/libmfem.a | grep ParaViewHDFDataCollection | head -3` lists at least one symbol.
 - `make seas_bp5_full -j 8` from `miniapps/seas` succeeds; `otool -L seas_bp5_full | grep hdf5` shows `libhdf5` from the `conda` env.
 - `make test-paraview-hdf-smoke` passes both serial and `mpirun -np 4`.
 - (Optional but recommended) ParaView 5.11+ opens `/tmp/test_paraview_hdf_smoke_out/smoke.vtkhdf` and renders the unit cube with the constant scalar field.
 - All Phase 1 tests still pass: `make test-fault-surface-vtu-binary test-fault-surface-vtu-gather-mpi test-fault-surface-vtu-continuity test-fault-surface-vtu-k4 test-fault-surface-vtu-field-filter`.
-

Phase 2b: Fault-submesh + VTKHDF writer integration (*local code work*)

This is the original Phase 2 implementation work, kept intact below the build-prerequisite gate. All requirements remain as specified.

Files to Create

- `miniapps/seas/io/fault_vtkhdf_writer.hpp` — wraps `mfem::ParaViewHDFDataCollection` for the fault submesh; consumes the same `LocalFaultPack` produced by Phase 1.
- `miniapps/seas/tests/unit/test_fault_surface_vtkhdf.cpp` — `#ifdef MFEM_USE_HDF5` test that round-trips a small fault dataset through HDF5 and verifies bit-exactness against the Phase 1 binary VTU.
- `miniapps/seas/tests/unit/test_fault_surface_vtkhdf_mpi.cpp` — `mpirun -np 4` test that all ranks write collectively into one HDF5 file and ParaView's HDF5 reader sees the merged result.

Files to Modify

- `miniapps/seas/io/paraview_output.hpp`:
 - Add `enum class FaultOutputMode { Vtu, Hdf5 }`; and a member `output_mode_` with setter `SetFaultOutputMode(FaultOutputMode)`.
 - When `output_mode_ == Hdf5`, `WriteFaultSurfaceVTU` redirects to the HDF5 writer (the function name is preserved for backward compat).
 - In the constructor, set `output_mode_ = Hdf5` by default IF the build defines `MFEM_USE_HDF5`; otherwise default to `Vtu`.
- `miniapps/seas/drivers/{tpv102,tpv104,tpv205,seas}_driver.cpp` (4 files):
 - Add `--paraview-fault-vtu` CLI flag that overrides the default and forces Phase 1's binary VTU output. Useful for debugging and for bisecting any HDF5-specific issues.
 - Add `--paraview-fault-hdf5` CLI flag (explicit opt-in, redundant with the default but provided for symmetry / explicit sbatch scripts).
 - On a non-HDF5 build, `--paraview-fault-hdf5` is rejected at parse time with: "This build does not support HDF5; rebuild with `MFEM_USE_HDF5=YES`. Falling back to binary VTU is automatic."
- `config/config.mk, config/defaults.mk`:
 - **No edit to the default values** — the build still defaults to `MFEM_USE_HDF5=NO` for users who don't need it. The plan adds documentation comments explaining how to enable it.
- `miniapps/seas/CLAUDE.md`:
 - Add a section "ParaView output mode" documenting that HDF5 is the production default (when the build enables it) and that the legacy per-rank ASCII path is gated by `--paraview-fault-legacy-ascii` (preserved through Phase 1 for debugging only).
- `miniapps/seas/jobs/*/sbatch` templates (the production sbatch templates only — not the dev ones):

- Bump the recommended `module load` line to include the parallel HDF5 module identified during the build prerequisite step.

Detailed Requirements

- Submesh construction** (one-time, in `InitFaultOutputBP5`):
 - Build a fault triangle submesh. For each interior fault face, emit one triangle in the submesh. For shared fault faces, emit one triangle owned by the local rank (Elem1-side, matching today's ownership convention in the VTU writer at line 731-735).
 - Construction approach: build a per-rank `Mesh` of triangle elements listing only the local fault triangles, then construct a `ParMesh` directly from per-rank `Mesh` plus a partition vector (every triangle on rank `r` is partitioned to rank `r`).
 - **Open question — investigate before implementation:** does MFEM's `ParSubMesh::CreateFromBoundary` (or a similar helper) accept the fault face list directly? See `fem/submesh/parsubmesh.hpp`. If yes, prefer that route — it handles ghost / shared-face bookkeeping automatically. If no, the per-rank-Mesh + partition route works.
- Field registration:**
 - Register 12+5 cell-data fields as L2-p0 `ParGridFunctions` on the submesh. Field names match the Phase 1 list exactly.
 - Each field is a single double per triangle (one element per face).
 - Static fields (`param_a`, `param_Dc`, `fault_x2`, `fault_x3`) are written once at the first save, then HDF5's per-step storage skips them on subsequent saves via `dc.SetFieldStaticForTimeSeries(name, true)` if such an API exists, else they re-emit each time (acceptable, ~1% overhead).
- Per-write update path:**
 - Reuse Phase 1's `BuildLocalFaultPack` to compute the per-face averages. Add a new helper `WriteFaultPackToParGFs(const LocalFaultPack&, std::vector<ParGridFunction*>&)` that scatters pack values into the submesh GFs.
 - Call `dc.SetCycle(cycle); dc.SetTime(time); dc.Save();`. This is a collective HDF5 write.
- HDF5 settings:**
 - `dc.SetCompression(true)` (zstd-style internal compression — does not require `MFEM_USE_ZLIB` per `fem/datacollection.hpp:673`).
 - `dc.SetCompressionLevel(3)` (fast).
 - `dc.SetDataFormat(VTKFormat::BINARY)` (default, but explicit).
 - `dc.SetHighOrderOutput(false)` — fault data is L2-p0, no high-order representation needed.
- Failure modes:**
 - Mid-run abort: HDF5 may leave the last cycle partially written. The file remains readable up to the last clean cycle by ParaView. Recovery: no action; ParaView truncates at the last good step.
 - Optionally call `H5Fflush(H5F_SCOPE_GLOBAL)` after each `Save` when `--paraview-fsync` is set (rank 0 only; cheap).

Interfaces

```
// New in seas::ParaViewOutput<MeshType>
enum class FaultOutputMode { Vtu, Hdf5 };

public:
    void SetFaultOutputMode(FaultOutputMode mode) { output_mode_ = mode; }
    FaultOutputMode GetFaultOutputMode() const { return output_mode_; }

private:
    FaultOutputMode output_mode_ =
#ifdef MFEM_USE_HDF5
        FaultOutputMode::Hdf5;    // default when HDF5 is built in
#else
        FaultOutputMode::Vtu;
```

```

#endif

#ifdef MFEM_USE_HDF5
    // Submesh + per-field GFs (lifetime-managed by ParaViewOutput)
    std::unique_ptr<ParMesh> fault_submesh_;
    std::unique_ptr<ParaViewHDFDataCollection> fault_hdf_dc_;
    std::vector<std::unique_ptr<ParGridFunction>> fault_hdf_gfs_;
    std::unique_ptr<L2_FECollection> fault_hdf_fec_;
    std::unique_ptr<ParFiniteElementSpace> fault_hdf_fes_;

    void InitFaultHdfOutput(); // once
    void WriteFaultHdfStep(int cycle, real_t time,
                          const vtul::LocalFaultPack &pack); // each save
#endif

```

Edge Cases to Handle

- **Build without HDF5:** `--paraview-fault-hdf5` is rejected at parse time. No `#ifdef` leakage into runtime decisions other than this CLI rejection.
- **HDF5 file conflicts on restart:** a re-run with the same `output_dir` overwrites any existing `fault_surface.vtkhdf`. Document this in `--help`. Restart-mode (`UseRestartMode(true)`) is a Phase 5+ follow-up.
- **Empty fault on a rank:** same as Phase 1. HDF5 collective writes with zero local elements are well-defined in MFEM's wrapper.
- **Field-filter (`SetFaultVTUFields`):** the filter applies in HDF5 mode too — fields not in the allow-list are not registered with the data collection, so they don't appear in the output file.
- **_k4 diagnostic fields:** registered conditionally on first call if `has_k4`; once registered, must be registered for every subsequent save. Document the invariant: `has_k4` must be all-on or all-off for the entire run.

Acceptance Criteria (Phase 2b — local code work)

- ☐ Phase 2a build prerequisites complete locally (above).
- ☐ BP5 1-yr microrun on the local Mac (`np=4` via `mpirun -np 4 seas_bp5_full ...` with a small mesh) produces exactly **1 file**: `output_dir/fault_surface.vtkhdf`.
- ☐ If ParaView 5.11+ is installed locally, it opens the file and renders all fields with time-series scrubbing. If not, `pyvista`'s `read("fault_surface.vtkhdf")` succeeds and `mesh.cell_data['slip_rate_strike'].shape` equals the expected face count.
- ☐ File size at fixed write count is < 50% of the corresponding Phase 1 binary VTU total (compression dominates).
- ☐ `test_fault_surface_vtkhdf.cpp` verifies bit-exactness vs Phase 1 binary VTU on the same input data, with `max-abs-diff < 1e-9` per cell per field.
- ☐ `test_fault_surface_vtkhdf_mpi.cpp` (`mpirun -np 4`) verifies that the merged HDF5 file contains every face from every rank exactly once.
- ☐ `make test-bp5-smoke` and `make test-tpv102` pass on the HDF5-enabled build.
- ☐ All existing sbatch scripts that use the SEAS drivers either (a) succeed with the new HDF5 default, or (b) opt into legacy via `--paraview-fault-vtu` with no behavior change.

Dependencies

- Depends on: Phase 1 (`LocalFaultPack` reused; binary VTU is the bit-exact reference).
- Depends on: Phase 2a (local HDF5-enabled build).
- Required by: Phase 2c (Frontera deploy).

Phase 2c: Replicate the build on Frontera and deploy (after 2a + 2b pass locally)

This sub-phase is the original “Frontera build steps” content but is deferred until 2a and 2b pass locally. No SEAS code changes happen here — only TACC build configuration plus a small verification run.

2c.1 Build prerequisite on Frontera

```
ssh frontera
cd /scratch2/10024/zhaochun/seas-project/seas-mfem
git fetch origin && git checkout <branch with Phase 1+2 changes>

# 1. Find the parallel HDF5 module. Frontera convention is
#   `phdf5/<version>` for the Intel-MPI parallel build. Verify that
#   `which h5pcc` resolves and that the underlying MPI matches the
#   impi19 already used by the existing sbatch headers.
module avail hdf5 2>&1 | grep -i hdf5
module spider phdf5

module load phdf5
echo "TACC_HDF5_DIR=$TACC_HDF5_DIR"
ls $TACC_HDF5_DIR/lib | head      # must list libhdf5.so / libhdf5_hl.so

# 2. Edit config/config.mk. Same diff as Phase 2a.4 except point
#   HDF5_DIR at $(TACC_HDF5_DIR) instead of $(CONDA_PREFIX):
#       MFEM_USE_HDF5   = YES
#       MFEM_USE_ZLIB   = YES
#       HDF5_DIR         = $(TACC_HDF5_DIR)
#       HDF5_OPT         = -I$(HDF5_DIR)/include
#       HDF5_LIB         = -L$(HDF5_DIR)/lib -lhdf5_hl -lhdf5
#   (zlib already on the link line via the TACC default toolchain;
#   if not, add -L$(TACC_ZLIB_DIR)/lib -lz to ZLIB_LIB.)

# 3. Rebuild MFEM and the seas miniapp. This is part of an idev
#   session or sbatch job - DO NOT run on the login node (15 min CPU
#   limit will kill the build).
idev -p small -N 1 -n 1 -t 02:00:00
make config
make clean && make -j 8 lib
cd miniapps/seas && make seas_bp5_full -j 8

# 4. Verify the build linked HDF5 correctly:
ldd seas_bp5_full | grep -i hdf5
# expected: libhdf5.so.* => /opt/apps/.../phdf5/.../lib/libhdf5.so.*
```

2c.2 Update production sbatch templates Edit miniapps/seas/jobs/bp5/bp5_system_update_500yr_strike_only. (and any other production templates) to add the parallel-HDF5 module load alongside the existing toolchain:

```
module load intel/19.1.1
module load impi/19.0.9
module load hypre/2.31.0
module load mumps/5.3
module load parmetis
module load petsc/3.15
+module load phdf5
```

No CLI changes needed in the sbatch — drivers default to HDF5 mode when the build defines MFEM_USE_HDF5. Existing `--paraview-fields slip_rate_strike` and other ParaView flags continue to work.

2c.3 Frontera microrun verification Submit a 1-yr BP5 run at np=100 (small queue, < 1 hr):

```
sbatch jobs/bp5/bp5_phase2c_microrun.sbatch # to be created;  
                                           # mirrors 500yr template  
                                           # but tfinal=3.16e7,  
                                           # 100 ranks, 1hr wall.
```

Phase 2c Acceptance gate

- ☐ Frontera build succeeds with MFEM_USE_HDF5=YES.
- ☐ Microrun produces **exactly 1 file** in `<results>/FaultSurface/: fault_surface.vtkhdf`.
- ☐ `du -sh <results>/FaultSurface/fault_surface.vtkhdf` < 100 MB at np=100, ~50 frames.
- ☐ `scp` or `rsync` of that single file from Frontera to laptop completes in < 2 minutes.
- ☐ ParaView opens the downloaded file and scrubs through all timesteps.
- ☐ No regression in existing tests: `make test-bp5-smoke && make test-tpv102` on Frontera.

Phase 2c Dependencies

- Depends on: Phase 2a + Phase 2b complete and green locally (HARD GATE).
 - Required by: Phase 2d (HDF5+ZFP) consumes the same ParaViewHDFDataCollection plumbing.
-

Phase 2d: ZFP lossy compression with separate bulk vs fault tolerances

Goal

After this phase, the production HDF5 path (Phase 2) optionally routes HDF5 chunks through the **H5Z-ZFP plugin** (filter id 32013, accuracy mode) rather than the default zlib/deflate filter. The bulk-volume collection (`pv_`, `pv_bulk_out`) and the fault collection (`fault_hdf_dc_`) carry **independent ZFP accuracy tolerances**, set by two separate CLI flags. This unlocks ~5–10× additional volumetric reduction for bulk wavefield output (TPV102/104/205) and ~3–5× for fault output, on top of Phase 2's lossless gains. Phase 2d is opt-in per run; the lossless deflate path from Phase 2 remains the default.

Background — what MFEM exposes today

Direct evidence from the current MFEM tree (`mesh/vtkhdf.cpp:105-119`):

```
H5Pset_chunk(dcpl, ndims, chunk);           // chunk size already correct  
if (compression_level >= 0)  
{  
    H5Pset_shuffle(dcpl);  
    H5Pset_deflate(dcpl, compression_level); // hard-wired to deflate  
}
```

The public API (`fem/datacollection.hpp:512-581, 648-684`) exposes only: `- SetCompression(bool) → toggles the deflate call above - SetCompressionLevel(int 0..9) → maps to deflate level - SetDataFormat, SetHighOrderOutput, UseRestartMode`

There is **no** public hook for any other H5Z filter, ZFP included. Phase 2d therefore needs a small, surgical patch into MFEM (one new enum, one new setter on `VTKHDF`, propagation through `ParaViewHDFDataCollection`). The patch is isolated and reversible — it does not change the default behaviour of any existing MFEM user.

Why per-collection (bulk vs fault) tolerance is natural

Bulk and fault outputs are already two distinct `ParaViewHDFDataCollection` instances (`pv_ / pv_bulk_out_ / a new fault_hdf_dc_ from Phase 2b). Each instance owns its own std::unique_ptr<VTKHDF> (created lazily in EnsureVTKHDF), so a per-instance setter touches only that instance's dcpl and does not leak across collections. No shared global state is required.`

This separation is **scientifically required**, not just a convenience:

Field group	Acceptable abs error	Reason
Bulk velocity	$\sim 1e-3$ m/s out of ~ 1 m/s peak	Wavefield visualization; sub-mm/s diff invisible in ParaView
Bulk stress	$\sim 1e3$ Pa out of $\sim 1e7$ Pa peak	4-decade dynamic range; 4 sig figs is plenty
Fault slip rate	$\sim 1e-12$ m/s out of $1e-9..1$ m/s	Spans 9 decades during interseismic→event; needs $\sim 1e-3$ RELATIVE
Fault state ψ	$\sim 1e-4$ absolute	Logarithm; small absolute error is fine
Fault traction	$\sim 1e2$ Pa out of $1e6..1e8$ Pa peak	4 sig figs sufficient for interpretation

Using a single global tolerance would either (a) over-compress fault slip rate during interseismic (lose 4–5 sig figs of a $1e-9$ m/s value) or (b) under-compress the bulk wavefield (waste 3–4 sig figs of unnecessary precision). Phase 2d therefore exposes two independent flags.

Sub-phase ordering

Sub-phase	What	Where	Acceptance gate
2d.1	Install H5Z-ZFP + zfp libraries; load plugin without MFEM changes; verify with a vanilla h5dump round-trip.	Local Mac	h5pcc test program writes/reads a ZFP-compressed dataset; h5ls -v shows filter 32013.
2d.2	Patch MFEM (mesh/vtkhdf.{hpp,cpp}) and fem/datacollection.{hpp,cpp}) to expose a CompressionAlgorithm selector + ZFP accuracy tolerance.	Local Mac	seas_test_vtkhdf_zfp round-trips a $1e6$ -cell L2-p0 GF with abs error $< \text{tol}$.
2d.3	Wire the new MFEM API into seas::ParaViewOutput<MeshType> for both bulk and fault paths; add CLI flags.	Local Mac	make test-fault-surface-vtkhdf-zfp and make test-bulk-vtkhdf-zfp pass; bit-similarity vs Phase 2 lossless within tolerance.

Sub-phase	What	Where	Acceptance gate
2d.4	Replicate the H5Z-ZFP build on Frontera; run a microrun with both bulk and fault using ZFP.	Frontera	One BP5 microrun with <code>--paraview-bulk-zfp-tol 1e-3</code> and <code>--paraview-fault-zfp-tol 1e-12</code> produces a single <code>fault_surface.vtkhdf</code> and per-cycle bulk VTKHDFs that scrub correctly in ParaView with the H5Z-ZFP plugin available.

Phase 2d.1: Install H5Z-ZFP and verify the plugin path

2d.1.1 Local install (mfem-dev conda env, macOS arm64)

```
conda activate mfem-dev
```

```
# zfp first (transitive dep of H5Z-ZFP)
conda install -y -c conda-forge "zfp=1.0.*"
# (or build from source: https://github.com/LLNL/zfp.git, cmake -DBUILD_SHARED_LIBS=ON)
```

```
# H5Z-ZFP plugin (HDF5 filter 32013). Conda-forge ships a build that
# matches conda-forge's HDF5; verify the HDF5 ABI matches by checking
# `otool -L $(conda info --base)/envs/mfem-dev/lib/plugin/libh5zzfp*.dylib`
conda install -y -c conda-forge "h5z-zfp"
```

```
# OR (build from source if the conda package mismatches HDF5):
# git clone https://github.com/LLNL/H5Z-ZFP.git && cd H5Z-ZFP
# mkdir build && cd build
# cmake -DCMAKE_INSTALL_PREFIX=$CONDA_PREFIX \
#       -DZFP_DIR=$CONDA_PREFIX -DHDF5_DIR=$CONDA_PREFIX ..
# make -j 8 install
```

```
# After install, the plugin must be discoverable by HDF5. Standard
# paths searched by libhdf5 are:
# $HDF5_PLUGIN_PATH (env var) → recommended; explicit
# /usr/local/hdf5/lib/plugin → unlikely on conda-forge
# {hdf5 install}/lib/plugin → likely under conda
export HDF5_PLUGIN_PATH=$CONDA_PREFIX/lib/plugin # or wherever it was installed
ls $HDF5_PLUGIN_PATH/libh5zzfp* 2>/dev/null # must exist
```

2d.1.2 Acceptance — vanilla HDF5 round-trip Write a tiny C program (NOT linked against MFEM) under `miniapps/seas/tests/unit/standalone/test_h5z_zfp_smoke.c`:

```
#include <hdf5.h>
#include <stdlib.h>
#include <stdio.h>
#define H5Z_FILTER_ZFP 32013

int main(void)
{
```

```

const hsize_t N = 1024 * 1024;
double *buf = malloc(N * sizeof(double));
for (hsize_t i = 0; i < N; ++i)
{ buf[i] = 1e-3 * (double)i + 1e-12 * (double)(i*i); }

hid_t f = H5Fcreate("/tmp/zfp_smoke.h5", H5F_ACC_TRUNC,
                  H5P_DEFAULT, H5P_DEFAULT);
hid_t s = H5Screate_simple(1, &N, NULL);
hid_t dcpl = H5Pcreate(H5P_DATASET_CREATE);
hsize_t chunk = 65536;
H5Pset_chunk(dcpl, 1, &chunk);
/* ZFP accuracy mode: cd_values per H5Z-ZFP API.
   0 => mode word, then mode-specific params.
   Easiest route: use the H5Z-ZFP CFP helpers (H5Pset_zfp_accuracy_cdata)
   OR pass the four-uint32 cd_values literal documented in the H5Z-ZFP
   README (mode=ACCURACY=3, then a double tolerance encoded as 2 uint32). */
/* Use the helper from H5Zzfp_props.h: */
#ifdef H5Z_ZFP_USE_PLUGIN
/* When using the loadable plugin, the helper headers may not be
   on the include path. In that case use the cd_values directly:
   cd[0]=major_version<<16/minor_version, cd[1]=mode (ACCURACY=3),
   cd[2..3]=tolerance bits. The standalone test should prefer the
   helper when available. */
extern int H5Pset_zfp_accuracy_cdata(double, size_t, unsigned int*);
unsigned int cd[10] = {0};
size_t cd_nelmts = 10;
H5Pset_zfp_accuracy_cdata(1e-3, cd_nelmts, cd);
H5Pset_filter(dcpl, H5Z_FILTER_ZFP, H5Z_FLAG_MANDATORY,
             cd_nelmts, cd);
#else
/* Sample explicit cd_values for ACCURACY=3, tol=1e-3
   (verify exact encoding against H5Z-ZFP/src/H5Zzfp_props.h). */
unsigned int cd[6] = { /* see README; NOT a literal here */ 0 };
H5Pset_filter(dcpl, H5Z_FILTER_ZFP, H5Z_FLAG_MANDATORY, 6, cd);
#endif
hid_t d = H5Dcreate2(f, "vals", H5T_IEEE_F64LE, s,
                   H5P_DEFAULT, dcpl, H5P_DEFAULT);
H5Dwrite(d, H5T_NATIVE_DOUBLE, H5S_ALL, H5S_ALL, H5P_DEFAULT, buf);
H5Dclose(d); H5Pclose(dcpl); H5Sclose(s); H5Fclose(f);

/* Read back and check abs error <= 1e-3. */
f = H5Fopen("/tmp/zfp_smoke.h5", H5F_ACC_RDONLY, H5P_DEFAULT);
d = H5Dopen2(f, "vals", H5P_DEFAULT);
double *rb = malloc(N * sizeof(double));
H5Dread(d, H5T_NATIVE_DOUBLE, H5S_ALL, H5S_ALL, H5P_DEFAULT, rb);
double max_err = 0.0;
for (hsize_t i = 0; i < N; ++i)
{ double e = rb[i] - buf[i]; if (e < 0) e = -e; if (e > max_err) max_err = e; }
H5Dclose(d); H5Fclose(f);
fprintf(stderr, "max abs err: %g (tol 1e-3)\n", max_err);
free(buf); free(rb);
return (max_err <= 1e-3) ? 0 : 1;
}

```

Build:

```
h5pcc -o test_h5z_zfp_smoke test_h5z_zfp_smoke.c # -lh5zzfp added by plugin loader
HDF5_PLUGIN_PATH=$CONDA_PREFIX/lib/plugin ./test_h5z_zfp_smoke
h5dump -p -A -d /vals /tmp/zfp_smoke.h5 | grep -i 'FILTERS\|ZFP\|32013'
```

Phase 2d.1 acceptance gate

- ☐ which h5pcc resolves; h5cc -showconfig lists ZFP as a known filter.
 - ☐ Standalone smoke writes a 1M-element ZFP-compressed dataset; max abs error $\leq 1e-3$.
 - ☐ h5ls -v /tmp/zfp_smoke.h5 (or h5dump -p) shows filter ID 32013 on the /vals dataset.
 - ☐ Compression ratio $> 4\times$ vs the uncompressed equivalent (1024×1024 doubles = 8 MB $\rightarrow$ ZFP ≤ 2 MB at tol= $1e-3$).
-

Phase 2d.2: Patch MFEM to expose a filter selector + ZFP tolerance

Files to Modify (in the seas-mfem tree at /Users/chunhuizhao/projects/seas-mfem):

- mesh/vtkhdf.hpp — add a CompressionAlgorithm enum and a parameter set; add EnableZfpAccuracy(double tol).
- mesh/vtkhdf.cpp — branch in EnsureDataset (lines 105–120) on the algorithm and call the appropriate H5Pset_* filter.
- fem/datacollection.hpp — add enum class HDFCompression { Deflate, ZfpAccuracy }, void SetHDFCompression(HDFCompression, double param = 0.0), and a const accessor.
- fem/datacollection.cpp — propagate the choice into VTKHDF inside TSave() (lines 1407–1418).
- config/defaults.mk and config/config.mk — add MFEM_USE_H5Z_ZFP flag and H5Z_ZFP_DIR for explicit linker path (optional — plugin path also works).

Detailed requirements:

1. **Enum** in mesh/vtkhdf.hpp (next to the existing EnableCompression/DisableCompression block):

```
enum class CompressionAlgorithm
{
    None,
    Deflate,           // existing default; uses compression_level
    ZfpAccuracy        // ZFP filter 32013, accuracy mode, abs-error tol
};

void SetCompressionAlgorithm(CompressionAlgorithm alg) { algorithm = alg; }
void SetZfpAccuracy(double tol)
{
    MFEM_VERIFY(tol > 0.0,
                "ZFP accuracy tolerance must be > 0");
    zfp_accuracy_tol = tol;
    algorithm = CompressionAlgorithm::ZfpAccuracy;
}
```

Add corresponding private members:

```
CompressionAlgorithm algorithm = CompressionAlgorithm::Deflate;
double zfp_accuracy_tol = 1e-3; // default; only used in ZfpAccuracy
```

Keep EnableCompression(int) and DisableCompression() working — they continue to control the deflate level when algorithm == Deflate.

2. **Filter dispatch** in mesh/vtkhdf.cpp:EnsureDataset, replacing lines 116–120:

```

if (algorithm == CompressionAlgorithm::Deflate &&
    compression_level >= 0)
{
    H5Pset_shuffle(dcpl);
    H5Pset_deflate(dcpl, compression_level);
}
else if (algorithm == CompressionAlgorithm::ZfpAccuracy)
{
#ifdef MFEM_USE_H5Z_ZFP
    // ZFP only compresses floating-point datasets. If `type` is not
    // F32/F64, fall back to deflate to avoid filter rejection.
    if (H5Tequal(type, H5T_IEEE_F64LE) > 0 ||
        H5Tequal(type, H5T_IEEE_F32LE) > 0)
    {
        constexpr unsigned H5Z_FILTER_ZFP = 32013;
        unsigned cd[10] = {0};
        size_t cd_nelmts = 10;
        // Encode ACCURACY mode with the helper from H5Zzfp_props.h.
        // Helper signature (from H5Z-ZFP):
        //   H5Pset_zfp_accuracy_cdata(double tol,
        //                               size_t cd_nelmts,
        //                               unsigned* cd_values);
        extern int H5Pset_zfp_accuracy_cdata(double, size_t, unsigned*);
        H5Pset_zfp_accuracy_cdata(zfp_accuracy_tol, cd_nelmts, cd);
        H5Pset_filter(dcpl, H5Z_FILTER_ZFP, H5Z_FLAG_MANDATORY,
                      cd_nelmts, cd);
    }
    else
    {
        // Integer/connectivity datasets: keep them lossless.
        if (compression_level >= 0)
        {
            H5Pset_shuffle(dcpl);
            H5Pset_deflate(dcpl, compression_level);
        }
    }
}
#else
    MFEM_ABORT("CompressionAlgorithm::ZfpAccuracy requested but MFEM was "
               "built without MFEM_USE_H5Z_ZFP.");
#endif
}
// else: CompressionAlgorithm::None → no filter applied

```

Critical invariant: ZFP must NOT be applied to integer datasets (connectivity, offsets, types). The branch on `H5Tequal` enforces this at the lowest level so all upstream callers (mesh, GFs, time metadata) get the right behaviour automatically.

3. ParaViewHDFDataCollection plumbing in fem/datacollection.{hpp,cpp}:

```

// datacollection.hpp, inside ParaViewHDFDataCollection (public)
enum class HDFCompression { Deflate, ZfpAccuracy };

/// Choose the HDF5 chunk filter for this collection.
/// `param` is the deflate level (0..9) when alg=Deflate, or the
/// ZFP accuracy tolerance (abs error, > 0) when alg=ZfpAccuracy.

```

```

void SetHDFCompression(HDFCompression alg, double param);

HDFCompression GetHDFCompression() const { return hdf_alg_; }
double GetHDFCompressionParam() const { return hdf_param_; }

private:
    HDFCompression hdf_alg_ = HDFCompression::Deflate;
    double          hdf_param_ = 6.0; // deflate level default

```

In `TSave()` (`datacollection.cpp:1407–1424`), replace the `EnableCompression/DisableCompression` block with:

```

switch (hdf_alg_)
{
case HDFCompression::Deflate:
    vtkhdf->SetCompressionAlgorithm(VTKHDF::CompressionAlgorithm::Deflate);
    if (hdf_param_ >= 0.0)
    { vtkhdf->EnableCompression(int(hdf_param_)); }
    else
    { vtkhdf->DisableCompression(); }
    break;
case HDFCompression::ZfpAccuracy:
    vtkhdf->SetZfpAccuracy(hdf_param_);
    break;
}

```

`SetCompression(bool)` continues to toggle deflate as today (preserves backward compat for any user not using the new API).

4. Build flag in `config/defaults.mk` and `config/config.mk`:

```

# Use H5Z-ZFP filter (filter id 32013) for lossy floating-point
# compression in VTKHDF output. Requires MFEM_USE_HDF5=YES and the
# H5Z-ZFP plugin to be installed (or linked statically). At runtime,
# set HDF5_PLUGIN_PATH to the directory containing libh5zzfp.{so,dylib}.
MFEM_USE_H5Z_ZFP = NO

# Optional explicit linker path; usually unnecessary if HDF5 plugin
# discovery is correctly configured at runtime.
H5Z_ZFP_DIR =
H5Z_ZFP_OPT = -I$(H5Z_ZFP_DIR)/include
H5Z_ZFP_LIB = -L$(H5Z_ZFP_DIR)/lib -lh5zzfp

```

5. Local build steps (after the patch):

```

cd /Users/chunhuizhao/projects/seas-mfem
sed -i '' 's/^MFEM_USE_H5Z_ZFP = NO/MFEM_USE_H5Z_ZFP = YES/' config/config.mk
make config
make clean && make -j 8 lib
cd miniapps/seas && make seas_bp5_full -j 8

# Verify symbol presence
nm lib/libmfem.a | grep -i ZfpAccuracy | head
nm miniapps/seas/seas_bp5_full | grep -i ZfpAccuracy | head

```

Phase 2d.2 acceptance gate

- ☐ `mesh/vtkhdf.hpp` exposes `CompressionAlgorithm` enum and the new setter.

- fem/datacollection.hpp exposes HDFCompression enum and SetHDFCompression.
- A unit test test_vtkhdf_zfp.cpp writes a 1e6-cell L2-p0 GF through ParaViewHDFDataCollection with SetHDFCompression(ZfpAccuracy, 1e-6), reads it back via pyvista (or h5py + vtk), and asserts max-abs-diff $\leq 1e-6$. Compression ratio $> 5\times$ the lossless variant on the same input.
- All Phase 2 tests still pass with MFEM_USE_H5Z_ZFP=YES (Deflate remains the default in MFEM; no behavioral regression).

Phase 2d.3: Wire the new MFEM API into seas, with separate bulk/fault flags

Files to Modify:

- miniapps/seas/io/paraview_output.hpp:
 - Plumb new setters that delegate to the underlying ParaViewHDFDataCollection:


```
void SetVolumeHDFCompression(ParaViewHDFDataCollection::HDFCompression alg,
                             double param);

#ifdef MFEM_USE_HDF5
void SetFaultHDFCompression(ParaViewHDFDataCollection::HDFCompression alg,
                             double param);

#endif
```

Volume setter forwards to pv_ IF pv_ is HDF (Phase 2 makes the bulk path also support HDF mode in a follow-up; for Phase 2d the bulk pv stays VTU UNLESS the driver constructed an HDF-backed ParaViewOutput). Fault setter forwards to fault_hdf_dc_ introduced in Phase 2b.
 - The output_mode_ == Vtu path silently ignores ZFP setters with a one-time rank-0 warning (“ZFP requested but the active fault writer is binary VTU; tolerance ignored”).
- miniapps/seas/drivers/{tpv102,tpv104,tpv205,seas}_driver.cpp:
 - Add CLI flags:
 - * --paraview-bulk-zfp-tol X (real, default 0 = disabled)
 - * --paraview-fault-zfp-tol X (real, default 0 = disabled)
 - * --paraview-bulk-deflate-level N (int 0..9, default -1 = MFEM default)
 - * --paraview-fault-deflate-level N (int 0..9, default -1)
 - Parse-time validation:
 - * On a build with MFEM_USE_H5Z_ZFP=NO, both *-zfp-tol flags with X > 0 are rejected at parse time with a clear error: “--paraview-{bulk,fault}-zfp-tol requires the seas-mfem build to define MFEM_USE_H5Z_ZFP=YES; current build has it disabled.”
 - * On a build with MFEM_USE_HDF5=NO, both *-zfp-tol and *-deflate-level flags are rejected.
 - * Mutually exclusive at the driver level: --paraview-bulk-zfp-tol > 0 AND --paraview-bulk-deflate-level >= 0 is rejected with “choose either ZFP-accuracy OR deflate, not both”. Same for fault.
 - Wire-up:


```
if (paraview_bulk_zfp_tol > 0.0)
{ pv_out->SetVolumeHDFCompression(
    ParaViewHDFDataCollection::HDFCompression::ZfpAccuracy,
    paraview_bulk_zfp_tol); }
else if (paraview_bulk_deflate_level >= 0)
{ pv_out->SetVolumeHDFCompression(
    ParaViewHDFDataCollection::HDFCompression::Deflate,
    double(paraview_bulk_deflate_level)); }
// else: leave default (deflate level 6)

if (paraview_fault_zfp_tol > 0.0)
{ pv_out->SetFaultHDFCompression(
    ParaViewHDFDataCollection::HDFCompression::ZfpAccuracy,
    paraview_fault_zfp_tol); }
```

- // else as above*
- `miniapps/seas/CLAUDE.md`:
 - Add a “ZFP lossy output” section recommending **defaults**: | Driver | `--paraview-bulk-zfp-tol` | `--paraview-fault-zfp-tol` | | | | | seas (BP5) | (no bulk PV by default) | `1e-12` (slip-rate floor) | | `tpv102 / 104 / 205` | `1e-3` (m/s for velocity) | `1e-12` | Document that **fault tolerance must be smaller** because slip-rate spans `1e-9..1e0` m/s; using bulk-style `1e-3` would erase 6 decades.
- `miniapps/seas/jobs/*/sbatch` production templates: add the two flags as commented-out documentation but do NOT enable by default (Phase 2 lossless remains the default until Phase 2d ships).

Edge cases: - **Driver only constructs a fault HDF collection (no bulk HDF):** `--paraview-bulk-zfp-tol` is silently a no-op (warned once). - **Driver constructs `pv_bulk_out` independently** (TPV102 line ~1666): Phase 2d adds a setter on the *secondary* `ParaViewOutput` instance for `pv_bulk_out` so its ZFP tolerance is configurable separately from the primary `pv_out`. - **HDF5 plugin not on `HDF5_PLUGIN_PATH` at run time:** `H5Pset_filter` succeeds (it is registered as filter 32013) but the actual compression call fails with `H5Z: required filter is not registered`. Detect this in `Save()` by wrapping in `H5Eset_auto` with a custom handler that emits a helpful message: “H5Z-ZFP plugin not found at runtime. Set `HDF5_PLUGIN_PATH` to the directory containing `libh5zzfp.{so,dylib}` and re-run.” Implement at the seas wrapper level (NOT in MFEM) so MFEM remains generic. - **ZFP applied to non-FP datasets:** handled by the F64/F32 type check inside the MFEM patch (Phase 2d.2 step 2). Connectivity stays lossless via deflate.

Acceptance criteria (Phase 2d.3): - [] `make test-fault-surface-vtkhdf-zfp` passes locally — writes a fault HDF with `--paraview-fault-zfp-tol 1e-12`, reads back via `pyvista`, asserts `max(|slip_rate_strike_zfp - slip_rate_strike_lossless|) ≤ 1e-12`. - [] `make test-bulk-vtkhdf-zfp` passes locally — same idea on bulk velocity field with `--paraview-bulk-zfp-tol 1e-3`. - [] `make test-paraview-tolerance-isolation` (NEW) — sets `bulk_zfp=1e-3` and `fault_zfp=1e-12`, runs a 1-yr BP5 microrun, asserts: (a) bulk velocity max-abs-diff vs lossless $≤ 1e-3$, (b) fault slip_rate_strike max-abs-diff vs lossless $≤ 1e-12$, (c) compression ratios are *different* between bulk and fault (proves the two settings are independent and not cross-talking). - [] `make test-bp5-smoke` && `make test-tpv102` still pass without ZFP flags (regression gate — default must be deflate). - [] On a build with `MFEM_USE_H5Z_ZFP=NO`, passing `--paraview-fault-zfp-tol 1e-12` aborts at parse time with the documented error string (verified by a shell test). - [] Manual ParaView 5.13+ open of the resulting `fault_surface.vtkhdf` shows correct geometry and field colormaps; a `pyvista` programmatic check matches.

Phase 2d.4: Frontera replication

```
ssh frontera
```

```
cd /scratch2/10024/zhaochun/seas-project/seas-mfem
```

```
# 1. Find or build H5Z-ZFP for the existing impi19 + Intel toolchain.
#   `module spider zfp` and `module spider h5z-zfp` first; if not
#   available as a TACC module, build from source under $WORK with
#   cmake -DCMAKE_INSTALL_PREFIX=$WORK/h5z-zfp ...
#   and set HDF5_PLUGIN_PATH=$WORK/h5z-zfp/lib/plugin in sbatch.

# 2. Edit config/config.mk: MFEM_USE_H5Z_ZFP=YES; rebuild MFEM under
#   idev (login-node 15 min CPU limit will kill a clean rebuild).

# 3. Update production sbatch:
#   export HDF5_PLUGIN_PATH=$WORK/h5z-zfp/lib/plugin
#   ibrun ./seas_bp5_full --paraview-fault-zfp-tol 1e-12 \
#   --paraview-bulk-zfp-tol 1e-3 ...
```

Acceptance criteria (Phase 2d.4): - [] Frontera microrun (1 yr, 100 ranks) with both ZFP flags produces a single `fault_surface.vtkhdf` AND bulk HDF outputs that are < 30% the size of the deflate-only Phase 2c microrun. - [] `du -sh` shows the targeted 5–10× reduction for bulk and 3–5× for fault, vs Phase 2c lossless. - [] Downloaded files open in workstation ParaView 5.13+ (with H5Z-ZFP plugin available client-side) and scrub through all timesteps with no missing data. - [] `make test-bp5-smoke` and `make test-tpv102` still pass on Frontera build with `MFEM_USE_H5Z_ZFP=YES`.

Phase 2d Dependencies

- Depends on: Phase 2a, 2b, 2c (HDF5 path must be in place). This phase is the **incremental upgrade** to the Phase 2 production baseline.
- Required by: nothing — Phase 2d is an opt-in storage-saver, not a prerequisite for any subsequent phase.

Phase 3: Adaptive-schedule snapshot cap and CLI overrides

Goal

After this phase, the user can constrain the **total number of fault VTU snapshots** for a run via a single CLI flag, and the schedule auto-coarsens the interseismic interval to honour the cap without aborting. Drivers gain CLI overrides for the three regime intervals so users can tune output cadence per run without recompiling.

Files to Create

- `miniapps/seas/tests/unit/test_paraview_schedule_cap.cpp` — exercises the cap with synthetic `V_max` sequences across all three regimes.

Files to Modify

- `miniapps/seas/io/paraview_output.hpp`:
 - Extend `AdaptiveSchedule` with one optional field `max_total_snapshots = 0` (0 means uncapped) and a method `RecomputeIntervalForCap(real_t time_to_end, int snapshots_so_far)` that returns the new `dt_interseismic` so that `snapshots_so_far + projected_remaining ≤ max_total_snapshots`.
 - At the top of `Save/ShouldWrite` (just after the regime decision, before the time-comparison gate), call `RecomputeIntervalForCap` when `max_total_snapshots > 0` and `current_regime_ == 0`.
 - Increment a private member `int total_snapshots_written_ = 0` whenever the writer commits a write (rank 0 only, but the field is symmetric on all ranks because schedule decisions are collective).
- `miniapps/seas/drivers/{tpv102,tpv104,tpv205,seas}_driver.cpp` (4 files):
 - Add `GetIntArg(argc, argv, "--paraview-max-snapshots", 0)` parse and pass to `pv_out->GetSchedule().ma`
 - Add `GetRealArg(argc, argv, "--paraview-coseismic-dt", default), --paraview-nucleation-dt, --paraview-interseismic-dt` flags that override the schedule's `dt_coseismic`, `dt_nucleation`, `dt_interseismic`.

Detailed Requirements

1. The cap **must not change the coseismic or nucleation cadences** — those are physically meaningful (we want every event well-resolved). The cap only inflates `dt_interseismic` when needed.
2. Cap calculation, given current time `t`, end time `T`, snapshots so far `k`, cap `K`, current regime intervals (`dt_co`, `dt_nuc`, `dt_inter`):

$$\text{remaining_budget} = K - k$$

```

if remaining_budget <= 0:
    dt_inter_new = T - t    // emit at most one more
else:
    dt_inter_new = max(dt_inter_user, (T - t) / remaining_budget)
return dt_inter_new

```

Note: this is intentionally only an upper bound on writes — coseismic and nucleation phases will write as many as the physics demands. In pathological cases (many events) the cap can be exceeded; we log a warning at the run summary.

3. CLI flag `--paraview-max-snapshots N` with $N=0$ (the default) preserves existing behavior. $N>0$ activates the cap. Recommend default for production BP5: **5000**, corresponding to ~1 GB of binary VTU at nominal fault size.
4. The three `--paraview-*-dt` flags accept floats; values ≤ 0 are ignored (default kept). Flag values get clipped to be > 0 with `MFEM_VERIFY`.

Interfaces

```

// New member of seas::ParaViewOutput<MeshType>::AdaptiveSchedule
struct AdaptiveSchedule {
    // ... existing fields ...
    int max_total_snapshots = 0;    // 0 = uncapped (default)
    real_t RecomputeIntervalForCap(real_t time_to_end,
                                   int snapshots_so_far) const;
};

// New member of seas::ParaViewOutput<MeshType>
private:
    int total_snapshots_written_ = 0;

```

Edge Cases

- `max_total_snapshots > 0` but the user already exceeded it via coseismic events: subsequent inter-seismic writes are spaced by $T - t$ (one final write) and a one-line warning prints once on rank 0.
- `max_total_snapshots = 1`: only the initial $t=0$ snapshot (which the drivers all emit unconditionally via `paraview_write(0, 0.0, V_ini)`) plus one more final. The schedule must not block $t=0$.
- Combination with `--paraview-every N` (step-based) or `--paraview-dt X` (fixed-dt): both take precedence over the adaptive cap, per the existing precedence in `Save` (lines 899–921).

Acceptance Criteria

- `test_paraview_schedule_cap.cpp` constructs synthetic sequences for $(\text{time}, V_{\text{max}})$ mimicking 1, 5, and 50 nucleation→event→interseismic cycles and verifies `total_snapshots_written_ ≤ K + n_events_in_run`.
- All four drivers parse the new flags; driving `--help` shows them.
- An end-to-end smoke run with `--paraview-max-snapshots 50` on a ~1 yr BP5 microrun produces ≤ 60 fault VTU files (50 + a few coseismic).

Dependencies

- Depends on: Phase 1 (writes are observable as integer increments of `total_snapshots_written_`). Phase 2 (HDF5) inherits the cap automatically because the schedule is owned by `ParaViewOutput`, not by the writer backend.
- Required by: nothing.

Phase 4: Decouple volume PV output from fault PV output

Goal

After this phase, drivers can request **fault-only** ParaView output (skipping the volume `pv_.Save()` cost and disk usage entirely), and the default for production quasidynamic runs becomes fault-only.

Files to Create

None.

Files to Modify

- `miniapps/seas/io/paraview_output.hpp`:
 - Add `bool emit_volume_save_ = true;` member, with setter `SetVolumeSaveEnabled(bool)`.
 - Inside `ForceSaveImpl`, gate `pv_.Save()` on `emit_volume_save_`.
 - The PVD/VTU bookkeeping (`fault_pvd_entries_`, `WriteFaultPVD`) is independent of `emit_volume_save_` and continues to fire.
- `miniapps/seas/drivers/{tpv102,tpv104,tpv205,seas}_driver.cpp`:
 - Replace the existing ad-hoc `--no-domain-pv` flag (currently only in one driver, per `tpv104_driver.cpp:484`) with a uniform `--no-volume-pv` flag wired into `pv_out->SetVolumeSaveEnabled(...)`.
 - Add `--volume-pv-dt X` flag for an independent cadence (overrides fault schedule for the volume save). When > 0 , use a separate `last_volume_write_time_` clock.

Detailed Requirements

1. The default for BP5 production (`seas_driver.cpp` for the BP5 path) becomes `--no-volume-pv` UNLESS `--volume-pv-dt X` is explicitly set. Rationale: BP5 visualization primarily uses the fault surface; the volume save is large (multiple GB per cycle) and is rarely opened.
2. For TPV102/104/205 (dynamic-rupture problems where the bulk wavefield IS scientifically interesting), keep the default as today (volume PV on at fault cadence, plus the existing optional `--paraview-bulk-dt`).
3. Document the new flag in each driver's `--help` text.

Interfaces

```
// New in seas::ParaViewOutput<MeshType>
public:
    void SetVolumeSaveEnabled(bool enabled) { emit_volume_save_ = enabled; }
    bool GetVolumeSaveEnabled() const { return emit_volume_save_; }

private:
    bool emit_volume_save_ = true;
    real_t last_volume_write_time_ = -1e30;    // independent volume clock
```

Edge Cases

- Driver passes `--no-volume-pv` AND `--volume-pv-dt X`: the explicit dt wins (volume save enabled with that cadence).
- Driver constructs `pv_bulk_out` (the second collection in `ParaView_bulk/`): unaffected — `pv_bulk_out` has its own `ParaViewOutput` instance and obeys its own flags.

Acceptance Criteria

- ☐ BP5 1-yr microrun with `--no-volume-pv` produces an empty `output_dir/ParaView/` (or no such directory) and a populated `output_dir/FaultSurface/` with the expected file count.
- ☐ Existing TPV102 sbatch scripts still get volume output (no regression).

- `make test-bp5-smoke` passes.

Dependencies

- Depends on: nothing (independent of Phase 1, but listed after for logical sequencing).
 - Required by: nothing.
-

Phase 5: Post-run packaging and download helpers

Goal

After this phase, a Python script `scripts/pack_fault_output.py` can ingest a `FaultSurface/` directory containing **legacy** per-rank ASCII VTUs and produce **either** a single `.vtkhdf`, **or** a chunked `tar.zst` suitable for fast `scp` to a workstation. This handles outputs already on disk that predate Phase 1.

Files to Create

- `miniapps/seas/scripts/pack_fault_output.py` — reads existing `fault_surface_r*_c*.vtu` files, optionally:
 - merges per-rank into single per-cycle files (Phase 1 layout retroactively), then `tar -cf` chunks of N cycles.
 - converts to a single VTKHDF5 file (if `h5py`, `vtk`, or `pyvista` available).
- `miniapps/seas/scripts/sbatch_pack_fault.sbatch` — example sbatch template that runs the script on a compute node within a 12 h allocation, including retry-on-failure and per-chunk completion detection.
- `miniapps/seas/scripts/download_fault.sh` — example client-side rsync wrapper with `--partial --partial-dir=.rsync-partial --info=progress2`.

Files to Modify

None.

Detailed Requirements

1. The packing script reads the existing `fault_surface.pvd` to enumerate (cycle, time) pairs, then groups files by cycle and merges per-rank arrays. The merger uses the same logic as Phase 1's gather (reading the ASCII VTUs, concatenating cell data and remapping triangle indices).
2. Output modes:
 - `--out-tar BASENAME` → emits `BASENAME_chunk0001.tar.zst`, `BASENAME_chunk0002.tar.zst`, ... each containing N=100 cycles. Idempotent: skips a chunk whose tar already exists and verifies on re-run.
 - `--out-hdf5 PATH.vtkhdf` → emits a single VTKHDF using `pyvista` or `vtk`'s Python bindings.
3. The sbatch template requests `-p small -N 1 -n 1 -t 12:00:00`, matches all SAFS sbatch conventions, and prints per-chunk progress.

Interfaces

None C++.

Edge Cases

- Truncated VTU files (from prior tar-on-login-node kills): the script must detect a malformed `<DataArray>` and skip the cycle with a warning.
- Mixed-version directories (some Phase 1 binary single-VTU, some legacy per-rank ASCII): the script supports both, recognising the layout by presence of `_r0_` substring in filenames.

Acceptance Criteria

- ☐ On the ~3.9M-file directory of job 7672120 (or its successor), the script produces ~40 chunks of ~100k files each in < 10 hours wall time inside a 12 h sbatch slot.
- ☐ Chunked tar mode is resumable: deleting a single chunk and re-running regenerates only that chunk.
- ☐ HDF5 mode produces a file < 2 GB that ParaView reads identically to scrubbing the original PVD.

Dependencies

- Depends on: nothing in the codebase (pure post-processing).
 - Required by: nothing. Optional for users who already have legacy outputs.
-

Testing Strategy

Per-phase

- Phase 1: 4 existing tests (continuity, k4, field-filter, adjacent-tri-uniformity) retargeted to read binary VTUs. 2 new tests: bit-exactness (serial) and gather correctness (`mpirun -np 4`).
- Phase 2: 2 new tests under `#ifdef MFEM_USE_HDF5`. `test_fault_surface_vtkhdf.cpp` verifies VTKHDF round-trip bit-exactness vs the Phase 1 binary VTU on identical input data. `test_fault_surface_vtkhdf_mpi.cpp` (`mpirun -np 4`) verifies merged single-file output. Manual ParaView visual check on a real BP5 microrun once tests pass.
- Phase 2d: 4 new tests under `#ifdef MFEM_USE_HDF5 && MFEM_USE_H5Z_ZFP`:
 - `test_vtkhdf_zfp.cpp` — pure MFEM test: write a 1e6-cell L2-p0 GF via `ParaViewHDFDataCollection::SetHDFCtol`, read back via `h5py` / `pyvista`, assert $\text{max-abs-diff} \leq \text{tol}$ AND compression ratio $\geq 4\times$.
 - `test_fault_surface_vtkhdf_zfp.cpp` — fault-side write through seas's `ParaViewOutput` with `--paraview-fault-zfp-tol`; bit-similar to Phase 2 lossless within tolerance.
 - `test_bulk_vtkhdf_zfp.cpp` — bulk-side write; same idea.
 - `test_paraview_tolerance_isolation.cpp` — sets bulk and fault to *different* ZFP tolerances in the same run; asserts (a) each field obeys its own tolerance and (b) the two compression ratios differ (proves no cross-talk).
- Phase 3: 1 new test exercising synthetic `V_max` sequences, asserting `total_snapshots_written_ ≤ K + n_events_observed`. Run separately on Phase 1 (VTU) and Phase 2 (HDF5) backends to confirm cap honoured by both.
- Phase 4: smoke tests on BP5 and TPV102 with each combination of `--no-volume-pv` and `--volume-pv-dt`.
- Phase 5: integration test against a small (~1k cycles × 4 ranks) ASCII legacy directory.

Cross-phase

- A regression test (`tests/integration/test_paraview_compaction.cpp`) runs the same BP5 1-yr microrun with the legacy ASCII writer (saved reference) and the new binary writer, and asserts that:
 - File count matches the predicted formula for each writer.
 - For each (cycle, field), max-abs-diff in the per-cell value is $< 1e-9$ (relative) — captures the ASCII↔binary representation transition.
 - `fault_surface.pvd` advances through the same set of cycles.

Verification on real benchmarks

Phase 1 only (binary VTU): - BP5 100-yr run on Frontera (`make seas_bp5_full` plus the existing sbatch template) at `np=400` with `--paraview-max-snapshots 1000`: - Expected file count: ≤ 1001 in `FaultSurface/`. - Expected total bytes: ≤ 5 GB. - Expected `tar -cf` time on a compute node: < 5 minutes. - Expected `scp` to laptop: < 15 minutes on a typical residential link (4 GB / 5 MB/s).

Phase 2 (HDF5, production target): - Same BP5 100-yr run at np=400 with `--paraview-max-snapshots 1000`: - Expected file count: **exactly 1** (`output_dir/fault_surface.vtkhdf`). - Expected total bytes: ≤ 2 GB (compression). - Expected scp to laptop: < 7 minutes on a typical residential link (2 GB / 5 MB/s). - Expected ParaView open time on workstation (one file, scrub through 1000 timesteps): < 30 s for first frame.

Phase 2d (HDF5 + ZFP, opt-in storage saver): - Same BP5 100-yr run at np=400 with `--paraview-max-snapshots 1000 --paraview-fault-zfp-tol 1e-12`: - Expected file count: **exactly 1** (`fault_surface.vtkhdf`). - Expected total bytes: ≤ 0.5 GB (4 $\times$ over Phase 2 lossless on fault fields, mostly slip & traction with $\sim 5e3$ cells $\times \sim 1000$ cycles). - Expected slip-rate max-abs-diff vs lossless reference: $\leq 1e-12$ m/s. - Same TPV102 1-second dynamic-rupture run at np=400 with `--paraview-bulk-zfp-tol 1e-3 --paraview-bulk-dt 0.1`: - Expected bulk-VTKHDF total size: 5–10 $\times$ smaller than Phase 2 lossless on the same cadence. - Expected velocity field max-abs-diff vs lossless: $\leq 1e-3$ m/s (visually indistinguishable in ParaView).

Risk Assessment

What could go wrong

0. **Phase 2d ZFP plugin not on HDF5_PLUGIN_PATH at runtime:** `H5Pset_filter` succeeds at dataset-create time (filter id is just an integer), but the actual write fails at first chunk flush with `H5Z: required filter is not registered`. **Mitigation:** at `Save()` entry on the first ZFP-enabled cycle, the seas wrapper performs a single throwaway dataset write with the same filter and inspects the HDF5 error stack; on failure, abort with a clear message naming `HDF5_PLUGIN_PATH` and the expected library name (`libh5zzfp.so` on Linux, `libh5zzfp.dylib` on macOS). Detect early — once per run, not per cycle.
- 0a. **Phase 2d numerical contamination:** a too-loose ZFP tolerance on a field that downstream analysis quantitatively depends on (e.g. computing seismic moment from `slip_rate_strike`) can silently bias results. **Mitigation:** the Phase 2d defaults table is conservative (1e-12 for fault, 1e-3 for bulk velocity); `--paraview-fault-zfp-tol` is documented in `--help` with an explicit warning that any value $> 1e-12$ MAY corrupt slip-rate integrals. Drivers print the chosen tolerances at startup and at the run summary so they appear in the SLURM job log alongside the results.
- 0b. **Phase 2d ZFP applied to integer datasets:** connectivity / offsets / types are integers, and ZFP rejects non-FP datatypes with H5 filter errors. **Mitigation:** the MFEM patch (`mesh/vtkhdf.cpp` filter dispatch) checks `H5Tequal(type, H5T_IEEE_F64LE)` and falls back to deflate for integer datasets. Verified by the `test_vtkhdf_zfp.cpp` unit test (which exercises a mesh save + GF save in the same file).
1. **Gather collective deadlock at np > 100:** an MPI rank that returns early from `WriteFaultSurfaceVTU` (e.g. on the legacy code path's `if (!has_fault_output_) return;` at line 587) does not call the gather, hanging the others. **Mitigation:** keep the early- return guarded by a collective `MPI_Allreduce` over `has_fault_output_`, or ensure the early return is consistent on all ranks (which today it is — `has_fault_output_` is set in `InitFaultOutputBP5` which is called by all ranks). Add an explicit comment.
2. **Field-name desync:** rank 0 has `_k4` fields enabled, other ranks don't. Detected by the `MPI_Bcast+hash` assertion in §Phase 1.3 above.
3. **HDF5 build slip:** a TACC user runs the new binary with a build that didn't enable `MFEM_USE_HDF5` and passes `--paraview-fault-hdf5`. **Mitigation:** clear runtime error; flag rejected at parse time on non-HDF5 builds.
4. **Binary VTU not readable by older ParaView:** ParaView ≤ 5.4 does not support `header_type="UInt64"`. **Mitigation:** header type is configurable; default `UInt64` for future-proof, fallback flag to `--paraview-fault-uint32` (4 GB cap, but compatible with older ParaView).
5. **Test-file representation lock-in:** the 4 existing unit tests parse VTU as ASCII regex. Switching the writer to binary breaks them. **Mitigation:** tests are explicitly listed in `Files to Modify` for Phase

1; their reader migrates to a small binary VTU parser that accepts both ASCII and binary appended formats.

6. **Numerical comparison tolerance:** the binary path stores full double precision; the ASCII path used `setprecision(10)`. Any test comparing bit-exact ASCII outputs across runs is now fragile. **Mitigation:** tests assert max-abs-diff tolerance (1e-9 for doubles, 1e-30 floor), not bit equality.

Known tricky areas in existing code

- `paraview_output.hpp:622-624` — `has_k4` is decided per-rank from the Size of three Vector arguments. All ranks must agree. Drivers currently pass either all three empty or all three populated; verify this invariant at the gather entry point.
- `paraview_output.hpp:728-737` — `if constexpr (std::is_same_v<MeshType, ParMesh>) ...` `#ifdef MFEM_USE_MPI ...`: keep this dual-guard pattern in the new code. Direct `mesh_.GetComm()` only inside both guards.
- `paraview_output.hpp:1086-1100 WriteFaultPVD` — the PVD is rewritten in full on every Save (truncating mode). This is $O(N_{\text{cycles}})$ per Save and on long runs (50,000+ cycles) becomes the dominant rank-0 write cost. After Phase 1 this should be addressed: append-only PVD emission using `std::ios::app` and one-pass header/footer rewrite. Track as a Phase 1 follow-up TODO; not blocking.

Open questions to resolve before implementation

These are deliberately **not** specified here because they require codebase investigation that should happen at implementation time, not at planning time:

1. **Phase 1:** Is there an existing MFEM utility (`VTKHelpers`, `AppendedDataStream`) that already does binary-appended VTU emission? If so, Phase 1 should call it instead of writing one from scratch. Search for `AppendedData` and `format="appended"` in `fem/`.
2. **Phase 2:** Can `ParSubMesh::CreateFromBoundary` (or a sibling) be built from a `Array<int>` of fault face indices, or do we need to construct a `ParMesh` of triangles manually with an explicit partition vector? See `fem/submesh/parsubmesh.hpp`.
3. **Phase 2:** What is the exact Frontera module name for the parallel HDF5 compatible with `intel/19.1.1 + impi/19.0.9` (the toolchain used by every existing sbatch template)? Run `module spider phdf5` during the build prerequisite step and amend the build instructions in §Phase 2.
4. **Phase 1:** The current PVD-rewrite-every-Save (`paraview_output.hpp:1086`) is probably already a perf problem at 50k cycles. Worth measuring before/after Phase 1 to decide whether to bundle the PVD-append fix. In Phase 2 (HDF5) the PVD goes away entirely, so this is a Phase 1-only concern.
5. **Phase 2d (RESOLVED 2026-04-29):** How does MFEM HDF5 integrate with ZFP today? → It does not. Direct evidence: `mesh/vtkhdf.cpp:118-119` hard-codes `H5Pset_shuffle + H5Pset_deflate(level)` and the public API (`fem/datacollection.hpp`) exposes only `SetCompression(bool)` / `SetCompressionLevel(int 0..9)` mapping to deflate. Chunking is already correct (`H5Pset_chunk` with 0.5 MB target, `mesh/vtkhdf.cpp:106-115`) so any H5Z filter — including ZFP filter id 32013 — can be plugged in by adding a filter selector at the single call site. Phase 2d.2 specifies that surgical patch. Bulk and fault tolerances are naturally separable because each `ParaViewHDFDataCollection` owns its own `VTKHDF` instance with its own `dcpl`, so per-collection settings do not cross-talk.

The implementation agent should investigate the unresolved items before starting the corresponding phase, and amend this plan with answers.

Baseline measurements (filled in during Phase 0)

Run	n_ranks	n_files	n_cycles	mean_vtu_bytes	total_dir_bytes	wall-time per Save
job 7672120	TBD	3,894,914	TBD	TBD	TBD	TBD
(small reference)	4	TBD	100	TBD	TBD	TBD

Acceptance for the plan as a whole

- ☐ Phases 1 and 2 are both merged (Phase 2 is the production target).
- ☐ The 250-yr BP5 production run, with the new HDF5 default, produces: - **exactly 1 fault output file** (`fault_surface.vtkhdf`); - ≤ 2 GB total fault-output disk usage; - downloadable to a workstation in < 10 minutes; - openable in ParaView 5.13+ directly with no client-side post-processing.
- ☐ On a build that does NOT enable HDF5, the same run gracefully falls back to Phase 1 binary VTU and produces $\leq 5,000$ files totalling ≤ 5 GB.
- ☐ Phase 2d (opt-in): with `--paraview-fault-zfp-tol 1e-12` AND `--paraview-bulk-zfp-tol 1e-3` on the same run, fault-output disk usage drops to ≤ 0.5 GB and bulk-output (when enabled) drops to 5–10% of Phase 2 lossless, with all numerical regression tests still passing within tolerance.
- ☐ The post-run packaging tool (Phase 5) is documented in `miniapps/seas/io/README.md` (creating a new file is fine if none exists).